

Homework for Spring WebFlux

Problem 1 - Virtual Threads

You have a standard Spring MVC application with the following blocking controller:

```
@GetMapping("/users/{id}")
public UserResponse getUser(@PathVariable Long id) {
    UserResponse user = userService.findById(id);
    OrderResponse order = orderService.findLatestOrder(id);
    return user.withLatestOrder(order);
}
```

Tasks:

- Enable virtual threads with a single config change

Problem 2 - Reactive REST API

Build a reactive REST API for a `Book` entity with fields `id`, `title`, `author`, and `price`.

Tasks:

- Create the full annotation-based WebFlux controller with GET all, GET by id, POST, PUT, DELETE
- GET all must support an optional `?author=` query param to filter results
- If a book is not found by id, return a proper 404 with a JSON error body, not an empty response
- Add a `/stream/books` endpoint that emits all books as a Server-Sent Event stream
- Validate that `title` and `author` are not blank and `price` is greater than zero, return 400 with field-level error messages on failure

Problem 3 - R2DBC (Database Layer)

Extend Problem 2 with a real PostgreSQL database using R2DBC.

Tasks:

- Write `schema.sql` for the `books` table with `created_at` and `updated_at` columns
- Create the `Book` entity with `@CreatedDate` and `@LastModifiedDate` and enable auditing
- Add a derived query method to find books by author
- Add a `@Query` method that finds all books where price is between a min and max value
- Make sure create checks for duplicate titles and returns a meaningful error if the title already exists

Problem 4 - WebClient (External Calls)

Your `BookService` needs to enrich each book response with author details fetched from a fictional external `AuthService` running at `https://api.authorservice.com`.

Tasks:

- Configure a `WebClient` bean for `AuthService` with a 3 second response timeout and connection timeout of 2 seconds
- Create an `AuthServiceClient` with a method `getAuthorDetails(String authorName)` returning `Mono<AuthorDetails>`
- Handle 404 from `AuthService` by returning a default `AuthorDetails` object with only the name populated instead of propagating the error
- Handle 5xx from `AuthService` by retrying up to 2 times with 300ms backoff before falling back to the default

Full Stack Reactive Project (Two Parts)

Build a reactive service called `LibraryService` that combines everything from this module. This is a two-part problem. Part one is the foundation. Part two adds the reactive and resilience layer on top.

• Problem 5

Build the complete backend for a `Library` that manages `Book` records and tracks `BorrowRecord` when a user borrows a book.

Schema:

```
books:          id, title, author, price, available (boolean), created_at, updated_at
borrow_records: id, book_id, user_id, borrowed_at, returned_at (nullable)
```

Requirements:

- Full CRUD for `Book` using the functional router model with `BookRouter` and `BookHandler`
 - `POST /api/v1/books/{id}/borrow?userId=` endpoint that:
 - Checks if the book exists, returns 404 if not
 - Checks if `available` is true, returns 409 Conflict with message "Book is not available" if not
 - Creates a `BorrowRecord` and sets `available = false` on the book in a single `@Transactional` reactive pipeline
 - `POST /api/v1/books/{id}/return?userId=` endpoint that:
 - Finds the open borrow record for this book and user where `returned_at` is null
 - Sets `returned_at = now()` and sets `available = true` on the book, again in a single transaction
 - All errors handled via `.onError()` on the router, not `@ControllerAdvice`
- **Problem 6**

Extend Part One with the following:

- Add a `GET /api/v1/books/stream` SSE endpoint that streams all currently available books from the database as a live feed
- Add a `WebClient` call to a fictional `UserService` at `https://api.userservice.com` to validate that the user exists before allowing a borrow. If `UserService` returns 404, return 400 Bad Request with message "User does not exist". If `UserService` is down (5xx), allow the borrow to proceed anyway as a fallback so the library does not go offline because of a user service outage

- Add a `GET /api/v1/books/{id}/profile` endpoint that uses `Mono.zip` to fetch the book from the database and the author details from a fictional `AuthService` simultaneously and returns a combined `BookProfileResponse`
- Add retry with 2 attempts and 500ms backoff on the `AuthService` call only